Hot Pots

5 instances · Download all inputs (.zip)

Hot Pots

You are given a rectangular grid. Some cells of the grid are grouped into disjoint “pots” (gray areas), each pot being a connected region of cells. On some cells of the grid there are numbered dumplings (circles).

You must move every dumpling so that it ends up inside some pot, according to the following rules. - Each dumpling moves along a straight line either horizontally or vertically. - A dumpling moves through empty cells and pots, and must stop as soon as it enters its "destination" pot; it cannot leave the grid. - The path of a dumpling cannot pass through any other dumpling, and paths of different dumplings cannot cross or share cells. - At the end, every dumpling must lie in a pot cell, and every pot must contain at least one dumpling. - Some pots have a number written in one of their cells. For each such pot, the sum of the numbers of all dumplings that end in that pot must be exactly equal to that number. - Pots without a number may contain any positive total sum of dumpling numbers, but still must contain at least one dumpling.

Goal

Find, for each dumpling, a direction and a number of steps describing a valid movement that satisfies all the above conditions.

Input Format

The input describes a single instance.

- The first line contains four integers, `R`, `C`, `N`, `A`, where:
  - `R` is the number of rows of the grid.
  - `C` is the number of columns of the grid.
  - `N` is the number of dumplings.

  - `A` is the number of pots.

- The next `N` lines describe the dumplings. The `i`-th of these lines contains three integers, `r_i`, `c_i`, `v_i`, where:
  - `(r_i, c_i)` is the initial position (row and column) of the `i`-th dumpling, with `1 <= r_i <= R` and `1 <= c_i <= C`.
  - `v_i` is the number written inside the `i`-th dumpling.
- The next line describes the target sums for the pots. It contains `A` integers `s_1, ..., s_A`, where:
  - `s_a > 0` means that pot `a` must contain dumplings whose numbers sum exactly to `s_a`.
  - `s_a = 0` means that pot `a` has no specified total; any positive sum is allowed, but pot `a` must still contain at least one dumpling.
- The next `R` lines describe the layout of pots on the grid. Each of these lines contains `C` integers. For row `r` and column `c`, the corresponding integer is:
  - `0` if cell `(r,c)` does not belong to any pot (it is an empty cell through which dumplings may move).
  - An integer `a` with `1 <= a <= A` if cell `(r,c)` belongs to pot `a`.

For each pot `a`, all cells marked with number `a` form one connected region via horizontal and vertical adjacency, and different pots are disjoint.

Output Format

You must output exactly `N` lines.

On the `i`-th line, print a movement command for the `i`-th dumpling in the following format:

- A character `U`, `D`, `L`, or `R` indicating the direction in which the dumpling is moved:
  - `U` = up (towards decreasing row index)
  - `D` = down (towards increasing row index)
  - `L` = left (towards decreasing column index)
  - `R` = right (towards increasing column index)

- A space.
- A non-negative integer `k` indicating how many steps the dumpling moves in that direction.

The meaning of a command `X k` for dumpling `i` is that the dumpling is shifted from its initial position `(r_i, c_i)` by exactly `k` cells in direction `X`, one cell at a time, so that:

- The final cell belongs to some pot (label in `{1,...,A}`).
- The paths of different dumplings do not cross or share intermediate cells, and no dumpling passes through the initial position of another dumpling.
- When all moves are applied simultaneously, all the rules described in the problem statement are satisfied.

If no solution exists, any output that does not satisfy the rules will be judged as incorrect.

Example

Instance in input:

```
5 5 5 4
1 1 5
2 3 2
3 5 2
4 4 4
5 3 1
4 0 6 0
0 0 1 1 1
0 0 0 0 0
0 2 2 2 0
0 0 0 0 0
3 3 0 0 4
```

Solution in output:

```
D 4
D 1
D 2
```

```
U 3
L 1
```

Visualizer

A graphical visualization can be obtained on ASP Chef.

Instances

Submit all instances at once

Upload one .zip containing a file per instance, each named after its instance number (e.g. `instance.1.out`, `2.txt`).

Browse... No file selected.

Submit all

Your code

Upload a .zip of the code your team used to generate the answers for this problem. Re-uploading replaces the previous file.

Current upload: download your code

Browse... No file selected.

Upload code

Instance 1 SOLVED ✓

▸ Show input

Paste your answer

Paste output here, or upload a file below

or upload a file

Browse... No file selected.

Submit instance 1

Instance 2 SOLVED ✓

▸ Show input

Paste your answer

Paste output here, or upload a file below

or upload a file

Browse... No file selected.

Submit instance 2

Instance 3 SOLVED ✓

▸ Show input

Paste your answer

Paste output here, or upload a file below

or upload a file

Browse... No file selected.

Submit instance 3

Instance 4 SOLVED ✓

▸ Show input

Paste your answer

Paste output here, or upload a file below

or upload a file

Browse... No file selected.

Submit instance 4

Instance 5 SOLVED ✓

▸ Show input

Paste your answer

Paste output here, or upload a file below

or upload a file

Browse... No file selected.

Submit instance 5